

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÁO CÁO HỌC PHẦN MỞ RỘNG
CÔNG NGHỆ PHẦN MỀM (CO300A)

Lớp: A01 - Nhóm 1
GVHD: Nguyễn Thành Công

STT	Họ và tên	MSSV	Tên lớp	Tên ngành
1	Nguyễn Văn Hùng	2311301	A01	Kỹ thuật máy tính
2	Trần Minh Trí	2313626	A01	Kỹ thuật máy tính
3	Nguyễn Tô Quốc Việt	2313898	A01	Kỹ thuật máy tính
4	Lê Công Vinh	2313912	A01	Kỹ thuật máy tính

THÀNH PHỐ HỒ CHÍ MINH, THÁNG 4 NĂM 2026



BẢNG PHÂN CHIA CÔNG VIỆC

STT	Họ và tên	MSSV	Đóng góp	Công việc cá nhân
1	Nguyễn Văn Hùng	2311301	100%	Implementation and Code Quality và Software Testing and Quality Assurance
2	Trần Minh Trí	2313626	100%	Process Model and Planning và Architecture Design
3	Nguyễn Tô Quốc Việt	2313898	100%	Software Testing and Quality Assurance và Reflection and Retrospective
4	Lê Công Vinh	2313912	100%	Requirements Engineering và System Modeling

Mục lục

Danh sách hình vẽ	3
1 Requirements Engineering	4
1.1 Stakeholders	4
1.2 Yêu cầu hệ thống	4
1.2.1 Functional Requirements - FR	4
1.2.2 Non-Functional Requirements - NFR	5
1.3 User Stories và Tiêu chí chấp nhận	5
1.4 Ưu tiên yêu cầu theo mô hình MoSCoW	6
1.5 Traceability Table	6
2 Process Model and Planning	6
2.1 Lựa chọn mô hình phát triển	6
2.2 Quản lý Backlog và Ước lượng thời gian	7
2.3 Theo dõi tiến độ	8
3 System Modeling	8
4 Architecture Design	11
4.1 Các thành phần và Đáp ứng NFR1	11
4.2 Đề xuất Công nghệ sử dụng	12
5 Implementation and Code Quality	12
5.1 Code Listing	12
5.2 Short Inspection Summary	13
5.3 Code Smells and Refactoring	14
5.3.1 Code Smell 1 - Long Method (create_event)	14
5.3.2 Code Smell 2 - Duplicate Logic (get_event reuse)	14
6 Software Testing and Quality Assurance	15
6.1 Các Functions / User Stories Testing	15
6.2 Test Plan	15
6.3 Test Cases	16
6.4 Test Automation Approach	16
7 Reflection and Retrospective	17
7.1 Team Retrospective: Start / Stop / Continue	17
7.2 Lessons Learned	18
7.3 Individual Reflections	18



Danh sách hình vẽ

1	Sequence Diagram (Create Event) mô tả cho việc tạo mới event	8
2	Sequence Diagram (EDit/Deltete Event) mô tả cho việc chỉnh sửa và xóa event .	9
3	State Diagram mô tả vòng đời các event	9
4	Class Diagram mô tả cách hiện thực các trong hệ thống này	10
5	Sơ đồ Kiến trúc phân tầng của hệ thống SERS	11

1 Requirements Engineering

Mục tiêu: Xác định các chức năng của hệ thống và những đối tượng sẽ tương tác với hệ thống Smart Event Reminder System (SERS).

1.1 Stakeholders

Hệ thống bao gồm các bên liên quan chính sau đây:

Stakeholder	Vai trò
User	Tạo và quản lý các sự kiện cá nhân.
Admin	Quản lý tài khoản người dùng và giám sát dữ liệu hệ thống.
Notification Service	Thành phần kỹ thuật chịu trách nhiệm kích hoạt và gửi nhắc nhở.
Project Team	Nhóm sinh viên chịu trách nhiệm thiết kế, lập trình và kiểm thử.
Instructor	Giảng viên đánh giá quy trình và chất lượng tài liệu kỹ thuật.

Bảng 1: Danh sách các bên liên quan của dự án SERS

1.2 Yêu cầu hệ thống

1.2.1 Functional Requirements - FR

- FR1:** Hệ thống cho phép người dùng tạo sự kiện mới.
- FR2:** Hệ thống cho phép người dùng nhập chi tiết sự kiện (tiêu đề, mô tả, ngày, giờ, địa điểm).
- FR3:** Hệ thống cung cấp chức năng chỉnh sửa thông tin các sự kiện đã tồn tại.
- FR4:** Hệ thống cho phép người dùng xóa các sự kiện khỏi danh sách quản lý.
- FR5:** Hệ thống tự động gửi nhắc nhở trước khi sự kiện bắt đầu theo thời gian đã cài đặt.
- FR6:** Hệ thống hiển thị thông báo nhắc nhở khi người dùng nhập sau thông tin khi tạo sự kiện.

1.2.2 Non-Functional Requirements - NFR

1. **NFR1 (Hiệu năng):** Thông báo nhắc nhở phải được gửi đến người dùng trong vòng tối đa 60 giây kể từ thời điểm kích hoạt.
2. **NFR2 (Tính khả dụng):** Giao diện người dùng phải trực quan, cho phép tạo một sự kiện trong ít hơn 3 phút thao tác.
3. **NFR3 (Độ tin cậy):** Dữ liệu sự kiện phải được lưu trữ an toàn và không bị mất khi ứng dụng khởi động lại hoặc mất kết nối mạng tạm thời.

1.3 User Stories và Tiêu chí chấp nhận

ID	User Story	Acceptance Criteria
US1	As a User, I want to create an event with a title, date, and time so that I can schedule my plans.	1. Hệ thống lưu sự kiện khi điền đủ thông tin bắt buộc. 2. Hiển thị thông báo lỗi nếu bỏ trống tiêu đề.
US2	As a User, I want to edit event details so that I can update my schedule if plans change.	1. Thông tin mới được cập nhật ngay lập tức vào cơ sở dữ liệu. 2. Người dùng có thể hủy chỉnh sửa mà không mất dữ liệu cũ.
US3	As a User, I want to delete an event so that I can remove cancelled activities.	1. Yêu cầu xác nhận trước khi xóa. 2. Sự kiện biến mất khỏi danh sách sau khi xóa thành công.
US4	As a User, I want to receive a reminder so that I am not late for my events.	1. Thông báo xuất hiện đúng thời gian quy định. 2. Nội dung thông báo hiển thị đúng tiêu đề sự kiện.
US5	As a User, I want to receive error notifications when entering invalid data so that I can ensure the event list remains accurate.	1. Hệ thống hiển thị thông báo lỗi cụ thể khi người dùng nhập sai định dạng dữ liệu. 2. Ngăn chặn việc lưu sự kiện vào danh sách nếu có trường dữ liệu không hợp lệ.

1.4 Ưu tiên yêu cầu theo mô hình MoSCoW

Phân loại	Tính năng / Yêu cầu
Must have	Tạo, sửa, xóa sự kiện; Chức năng gửi nhắc nhở tự động.
Should have	Nhập chi tiết địa điểm; Xem danh sách sự kiện theo lịch.
Could have	Mời người dùng khác; Chia sẻ sự kiện qua mạng xã hội.
Won't have	Đồng bộ hóa với Google Calendar/Outlook (trong phiên bản này).

Bảng 3: Bảng phân loại mức độ ưu tiên MoSCoW

1.5 Traceability Table

Mã FR	Mã User Story	Biểu đồ liên quan
FR1, FR2	US1	Sequence Diagram (Create Event), Class Diagram, State Diagram
FR3, FR4	US2	Sequence Diagram (Edit/Delete Event), Class Diagram, State Diagram
FR5	US4	State Diagram, Class Diagram
FR6	US5	Class Diagram

Bảng 4: Traceability Table thể hiện liên kết giữa Yêu cầu, User Story và Diagram

2 Process Model and Planning

2.1 Lựa chọn mô hình phát triển

- Mô hình được chọn: Scrum (Agile).
- Lý do nhóm chọn mô hình này:
 - *Phù hợp với tính chất dự án:* SERS yêu cầu phát triển nhanh các tính năng cốt lõi (tạo, sửa, xóa sự kiện) và có thể linh hoạt mở rộng. Việc áp dụng Scrum với các Sprint ngắn (2 tuần) giúp nhóm làm việc tập trung và dễ dàng đối phó với những thay đổi về thiết kế (ví dụ: cập nhật thêm class NotificationService hay Reminder).

- *Tối ưu hóa nguồn lực nhóm 4 người*: Quy trình Scrum phân chia task minh bạch, giúp 4 thành viên dễ dàng phối hợp giữa việc vẽ Diagram, lập trình logic và kiểm thử (Testing) mà không bị chồng chéo công việc.

2.2 Quản lý Backlog và Ước lượng thời gian

Nhóm sử dụng kỹ thuật **Planning Poker** để đánh giá độ phức tạp của từng User Story (US).

Product Backlog (Đồng bộ với MoSCoW)

ID	Hạng mục (User Story)	Độ ưu tiên	Story Points	Trạng thái
US1	Tạo sự kiện mới với các thông tin (tiêu đề, ngày, giờ).	Must have	5	Sprint 1
US2	Chỉnh sửa và cập nhật chi tiết sự kiện đã có.	Must have	8	Sprint 1
US3	Xóa sự kiện khỏi danh sách.	Must have	3	Sprint 1
US4	Hệ thống tự động gửi thông báo nhắc nhở.	Must have	13	Sprint 2
US5	Mời người dùng khác tham gia sự kiện.	Could have	8	Backlog
US6	Xem danh sách sự kiện theo lịch trình.	Should have	5	Backlog

Bảng 5: Danh sách Product Backlog tổng thể

Sprint Backlog (Sprint 1 - Kéo dài 2 tuần)

Mục tiêu Sprint: Hoàn thiện logic nền tảng (CRUD) và vòng đời của sự kiện. Để giám sát chặt chẽ, nhóm phân rã công việc bám sát vào các Diagram đã thiết kế:

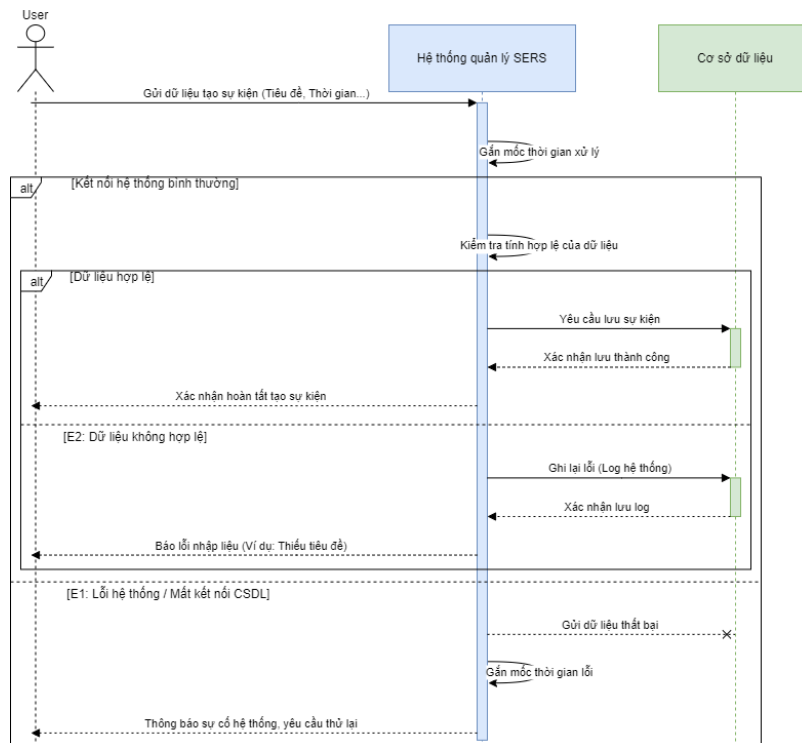
- **Task 1.1 [US1, US2]:** Hiện thực cấu trúc dữ liệu cho các class User, Event, Reminder dựa trên **Class Diagram (Hình 4)**. (3 SP)
- **Task 1.2 [US1]:** Xây dựng luồng xử lý tạo sự kiện mới, bao gồm logic kiểm tra tính hợp lệ dữ liệu theo **Sequence Diagram (Hình 1)**. (5 SP)
- **Task 1.3 [US2, US3]:** Hiện thực chức năng chỉnh sửa và xóa sự kiện theo **Sequence Diagram (Hình 2)**. (5 SP)
- **Task 1.4 [US1, US2]:** Quản lý chuyển đổi trạng thái của Event (Draft → Scheduled → Editing...) bám sát **State Diagram (Hình 3)**. (1 SP)
- **Task 1.5 [US1, US3]:** Viết Unit Test và refactor mã nguồn. (2 SP)

2.3 Theo dõi tiến độ

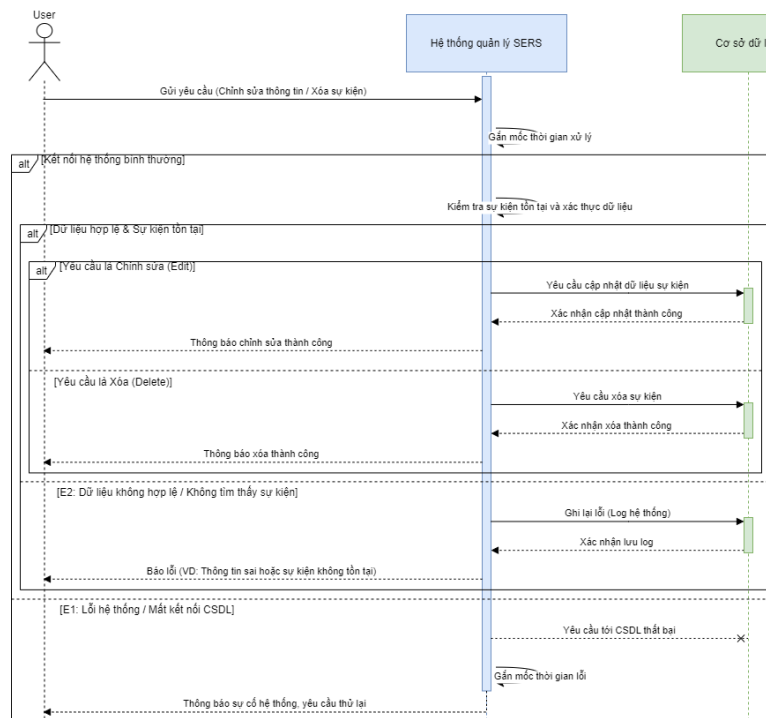
Thời gian	SP Dự kiến	SP Thực tế	Ghi chú tiến độ
Ngày 1	16	16	Họp Sprint Planning, phân rã task theo UML.
Ngày 4	13	13	Hoàn thành Task 1.1 (Khởi tạo các Class lỗi).
Ngày 8	8	8	Xong Task 1.2 (Pass luồng tạo Event - Hình 1).
Ngày 12	3	3	Xong Task 1.3 & 1.4 (Sửa/Xóa Event & Cập nhật State).
Ngày 14	0	0	Hoàn thành Unit Test (Task 1.5). Kết thúc Sprint 1.

Bảng 6: Tiến độ thực hiện Sprint 1

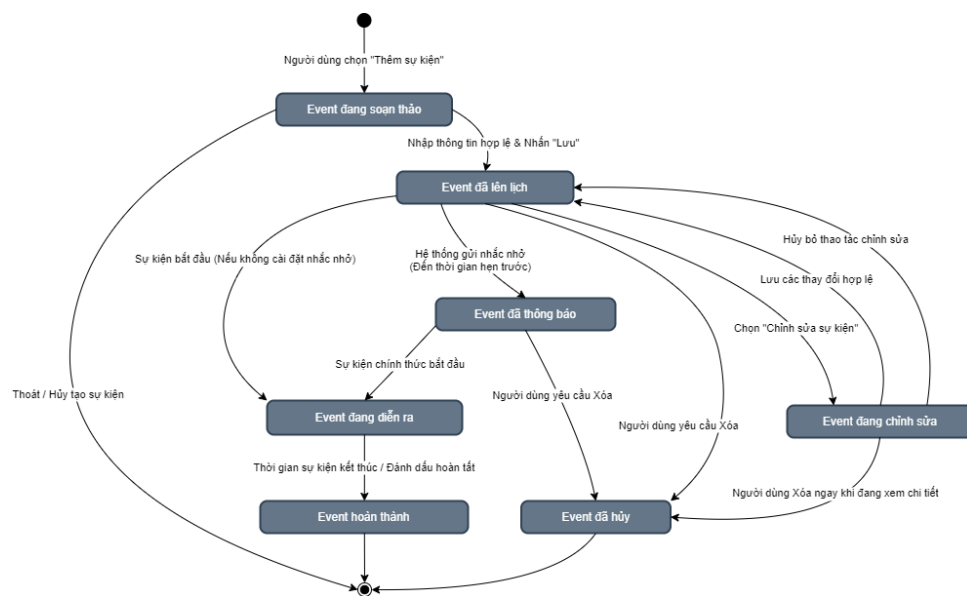
3 System Modeling



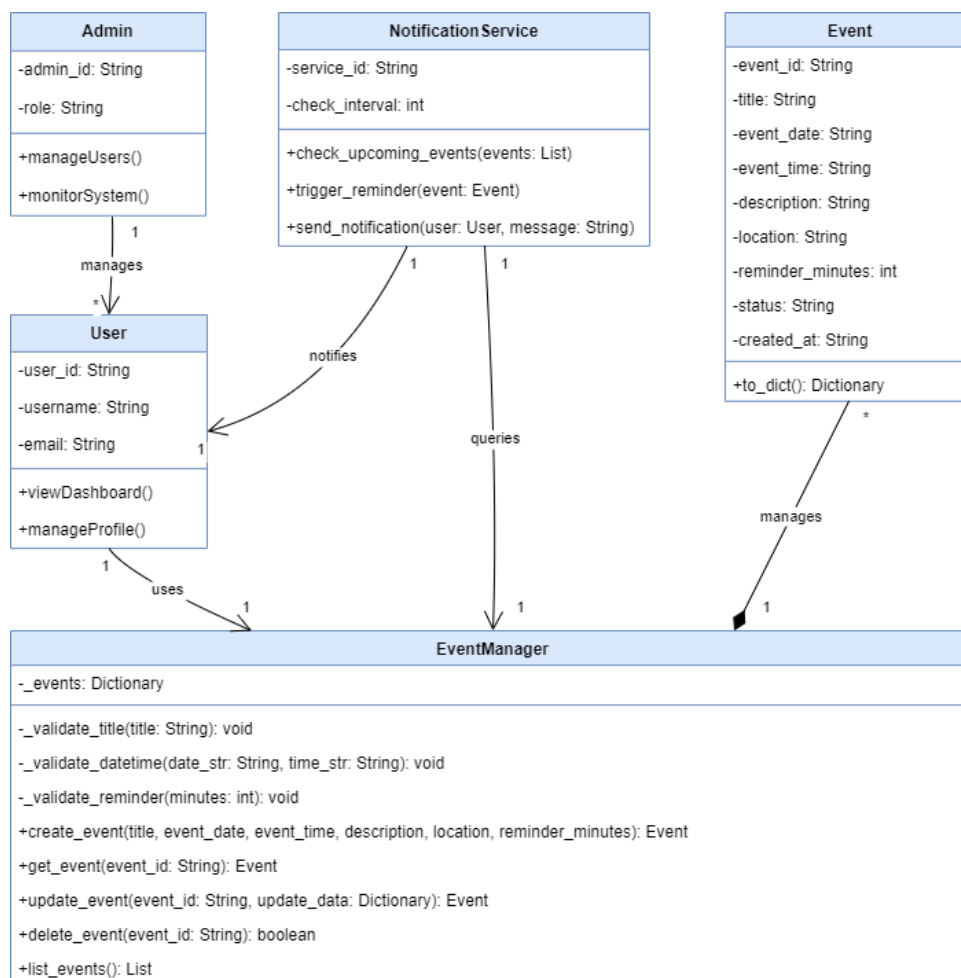
Hình 1: Sequence Diagram (Create Event) mô tả cho việc tạo mới event



Hình 2: Sequence Diagram (EDit/Delete Event) mô tả cho việc chỉnh sửa và xóa event

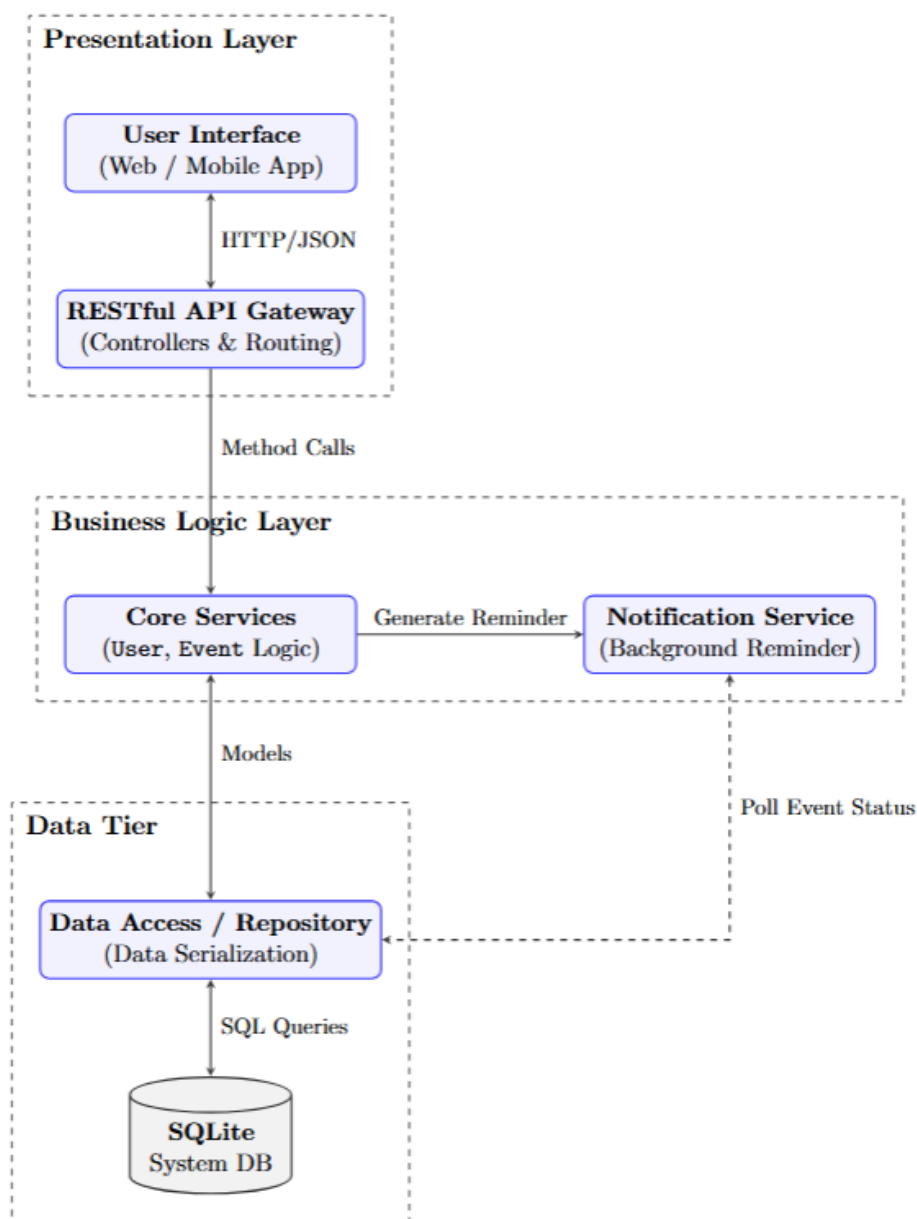


Hình 3: State Diagram mô tả vòng đời các event



Hình 4: Class Diagram mô tả cách hiện thực các trong hệ thống này

4 Architecture Design



Hình 5: Sơ đồ Kiến trúc phân tầng của hệ thống SERS

4.1 Các thành phần và Đáp ứng NFR1

1. **Presentation Layer (Tầng Giao diện):** Nhận thao tác từ người dùng và chuyển tiếp xuống hệ thống (tương ứng với mũi tên "Gửi yêu cầu" trong Hình 1 và 2).
2. **Business Logic Layer (Tầng Nghiệp vụ):** Dựa theo Class Diagram (Hình 4), tầng này chứa:

- **Core Services (User, Event):** Chịu trách nhiệm thực thi các CRUD logic, kiểm tra dữ liệu hợp lệ và chuyển đổi các trạng thái (EventStatus).
- **Notification Service (Đáp ứng NFR1):** Để đảm bảo thông báo gửi trong vòng 60 giây (NFR1), class NotificationService được chạy như một tiến trình ngầm (Background Service). Nó thực thi hàm checkPendingReminders() liên tục truy vấn Data Tier (với chu kỳ quét ngắn khoảng 30s) để triggerReminder() kịp thời.

3. **Data Tier (Tầng Dữ liệu):** Lưu trữ toàn bộ thông tin của Hệ thống quản lý SERS, trả kết quả truy vấn ("Xác nhận lưu thành công", "Xác nhận xóa thành công") lên tầng nghiệp vụ.

4.2 Đề xuất Công nghệ sử dụng

- **Backend: FastAPI (Python).** Khung làm việc này nhẹ, hỗ trợ xử lý bất đồng bộ (async), rất phù hợp để hiện thực hóa các class trong Hình 4.
- **Background Task: APScheduler.** Công cụ hoàn hảo để chạy class NotificationService ngầm dưới nền, đáp ứng việc check thời gian liên tục mà không làm chậm các API tạo/sửa sự kiện.
- **Database: SQLite.** Tích hợp sẵn trong Python, đủ mạnh để lưu trữ dữ liệu dự án môn học.

5 Implementation and Code Quality

5.1 Code Listing

Link source code: https://github.com/vinhle275/Software_Engineering_Extended

5.2 Short Inspection Summary

Tiêu chí	Mô tả
Đặt tên rõ ràng	Các lớp (Event, EventManager), hàm (create_event, delete_event) và biến (event_date, reminder_minutes) được đặt tên có ý nghĩa, thể hiện rõ mục đích sử dụng.
Thụt lề nhất quán	Tuân theo chuẩn PEP 8: thụt lề 4 khoảng trắng; sử dụng dòng trống để phân tách các khối logic.
Chú thích và docstring	Mỗi lớp và các hàm công khai đều có docstring; comment được dùng để giải thích các logic không rõ ràng.
Nguyên lý trách nhiệm đơn	Lớp Event chỉ chứa dữ liệu; EventManager xử lý toàn bộ logic CRUD và kiểm tra dữ liệu.
Kiểm tra đầu vào	Sử dụng các hàm private riêng (_validate_title, _validate_datetime, _validate_reminder) giúp hàm create_event rõ ràng hơn.

Bảng 7: Đánh giá chất lượng mã nguồn

ID	Loại vấn đề	Vấn đề phát hiện	Cách khắc phục đề xuất
I-01	Naming	Dict _events dùng tên quá chung (e dict)	Đổi thành _event_store hoặc _event_registry để rõ nghĩa hơn
I-02	Missing check	create_event không dùng strip() cho description trước khi lưu	Thêm description = description.strip() trước khi tạo Event
I-03	Readability	to_dict() không bao gồm trường created_at, không nhất quán với constructor	Thêm 'created_at': self.created_at vào dict trả về
I-04	Missing check	reminder_minutes không có giới hạn trên; giá trị rất lớn vẫn được chấp nhận	Thêm kiểm tra giới hạn trên (ví dụ: <= 10080 cho tối đa 1 tuần)
I-05	Documentation	list_events() thiếu docstring mô tả kiểu trả về	Thêm annotation kiểu trả về List[Event] và bổ sung docstring

Bảng 8: Kết quả peer-review

5.3 Code Smells and Refactoring

5.3.1 Code Smell 1 - Long Method (create_event)

Vấn đề:

Hàm `create_event` gọi ba hàm kiểm tra (validation) riêng biệt. Dù mỗi hàm đều nhỏ, nhưng khi hệ thống mở rộng, nhiều kiểm tra khác (ví dụ: kiểm tra sự kiện trùng lặp, kiểm tra định dạng địa điểm) sẽ tiếp tục được thêm trực tiếp vào `create_event`, khiến hàm này trở nên dài và khó quản lý.

Refactoring:

Tách ra một hàm duy nhất `_validate_all(title, date, time, reminder)` để gọi tất cả các hàm kiểm tra con. Khi đó, `create_event` chỉ cần gọi `_validate_all(...)` để giữ độ dài hàm dưới 10 dòng. Cách này tuân theo mẫu refactoring *Extract Method*.

```
# Before (smell)
def create_event(self, title, date, time, ...):
    self._validate_title(title)
    self._validate_datetime(date, time)
    self._validate_reminder(reminder_minutes)
    # 10+ more lines

# After (refactored)
def _validate_all(self, title, date, time, reminder):
    self._validate_title(title)
    self._validate_datetime(date, time)
    self._validate_reminder(reminder)

def create_event(self, title, date, time, ...):
    self._validate_all(title, date, time, reminder_minutes)
    new_event = Event(...)
    self._events[new_event.event_id] = new_event
    return new_event
```

5.3.2 Code Smell 2 - Duplicate Logic (get_event reuse)

Vấn đề:

Cả `delete_event` và các hàm trong tương lai như `update_event` đều cần lấy sự kiện theo ID. Nếu logic tìm kiếm (kiểm tra tồn tại key, ném lỗi `KeyError`) được viết trực tiếp trong từng hàm, nó sẽ bị lặp lại.

Refactoring:

Hàm `get_event` hiện tại đã gom toàn bộ logic này. Cách cải thiện là đảm bảo mọi hàm cần truy cập sự kiện đều gọi `get_event(event_id)` thay vì truy cập trực tiếp `self._events[event_id]`. Điều này tuân theo nguyên lý *DRY (Don't Repeat Yourself)*.

```
# Before (smell) - direct dict access in delete_event
def delete_event(self, event_id):
    if event_id not in self._events: # duplicated check
        raise KeyError(...)
    del self._events[event_id]

# After (refactored) - reuse get_event
def delete_event(self, event_id):
    event = self.get_event(event_id) # centralized check
    del self._events[event_id]
    return True
```

6 Software Testing and Quality Assurance

6.1 Các Functions / User Stories Testing

- **US-1:** Là người dùng, tôi muốn tạo một sự kiện để có thể theo dõi các hoạt động sắp tới.
- **US-2:** Là người dùng, tôi muốn nhận thông báo lỗi khi nhập dữ liệu không hợp lệ để đảm bảo danh sách sự kiện luôn chính xác.
- **US-3:** Là dịch vụ thông báo, tôi muốn các nhắc nhở được kích hoạt đúng thời điểm để người dùng được thông báo đúng lịch.

6.2 Test Plan

Thuộc tính	Chi tiết
Mục tiêu	Xác minh rằng SERS tạo sự kiện đúng với dữ liệu hợp lệ, từ chối dữ liệu không hợp lệ với thông báo lỗi rõ ràng, và kích hoạt nhắc nhở đúng thời điểm đã lên lịch.
Mức kiểm thử	Unit test cho các phương thức của EventManager; System test cho toàn bộ luồng gửi nhắc nhở.
Phương pháp kiểm thử	Black-box để kiểm tra tính đúng đắn chức năng; White-box (luồng xóa) để xác minh trạng thái nội bộ sau khi thực hiện.
Môi trường kiểm thử	Python 3.11 với framework unittest; sử dụng mock datetime để kiểm tra thời gian nhắc nhở.
Tiêu chí đạt	Tất cả đầu ra mong đợi khớp với kết quả thực tế; không có ngoại lệ chưa được xử lý.

Bảng 9: Kế hoạch kiểm thử hệ thống SERS

6.3 Test Cases

Test ID	Mô tả	Input	Kết quả mong đợi	Loại test	Mức
T1	Tạo sự kiện với dữ liệu hợp lệ	title="Team Meeting" date="2026-05-10" time="09:00"	Trả về đối tượng Event với các trường chính xác; sự kiện được lưu trong manager	Black-box	Unit
T2	Tạo sự kiện với tiêu đề rỗng	title="" date="2026-05-10" time="09:00"	Phát sinh ValueError: "Event title cannot be empty."	Black-box	Unit
T3	Tạo sự kiện với ngày trong quá khứ	title="Past Event" date="2020-01-01" time="08:00"	Phát sinh ValueError: "Event must be scheduled in the future."	Black-box	Unit
T4	Xóa sự kiện tồn tại	event_id hợp lệ từ T1	Trả về True; sự kiện không còn trong list_events()	White-box	Unit
T5	Kích hoạt nhắc nhở đúng thời điểm	event_time="10:00" now="09:45" reminder=15 min	Thông báo được kích hoạt chính xác lúc 09:45	Black-box	System

Bảng 10: Danh sách test case

6.4 Test Automation Approach

Các test case ở trên có thể được tự động hóa bằng cách sử dụng framework unittest tích hợp sẵn của Python hoặc pytest. Bên dưới là một ví dụ kiểm thử tự động tiêu biểu cho T1 và T2.

Việc chạy pytest trong quy trình CI/CD (ví dụ: GitHub Actions) sau mỗi lần commit giúp phát hiện sớm các lỗi hồi quy (regression) và tự động ghi lại kết quả kiểm thử. Báo cáo độ bao phủ kiểm thử (test coverage) có thể được tạo bằng pytest-cov để theo dõi những dòng mã đã được thực thi.

7 Reflection and Retrospective

Vào tối thứ Tư ngày 06/05/2026, nhóm tổ chức một buổi họp ngắn để nhìn lại toàn bộ quá trình thực hiện dự án Smart Event Reminder System (SERS). Mục tiêu của buổi họp không chỉ là kiểm tra phần nào đã hoàn thành, mà còn để mỗi thành viên nêu lại góc nhìn của mình về cách nhóm chia việc, phối hợp, xử lý phạm vi và giữ sự nhất quán giữa các phần trong báo cáo. Vì đây là một dự án thiên về quy trình công nghệ phần mềm hơn là xây dựng một sản phẩm hoàn chỉnh, nhóm tập trung đánh giá cách các phần Requirements, Planning, Modeling, Architecture, Implementation và Testing liên kết với nhau. Phần retrospective dưới đây được trình bày theo khung Start / Stop / Continue để rút ra những điểm cần cải thiện cho các dự án sau.

7.1 Team Retrospective: Start / Stop / Continue

Category	Team Reflection
Start	Nhóm nên bắt đầu thống nhất phạm vi, template trình bày và mối liên hệ giữa các phần ngay từ giai đoạn đầu. Sau mỗi mốc chính, nhóm cũng nên có một bước review ngắn để kiểm tra xem requirements, UML diagrams, architecture, code sample và test cases có đang khớp với nhau hay không. Điều này giúp phát hiện sớm những chỗ lệch trước khi ghép báo cáo cuối cùng.
Stop	Nhóm nên hạn chế việc mỗi thành viên làm phần của mình quá độc lập trong thời gian dài mà không cập nhật tiến độ cho các thành viên khác. Cách làm này có thể khiến nội dung giữa các phần bị lệch, hoặc một thành viên vô tình làm luôn phần việc của người khác. Nhóm cũng nên tránh để việc chỉnh sửa định dạng và kiểm tra tính nhất quán dồn vào cuối deadline.
Continue	Nhóm nên tiếp tục chia công việc theo thế mạnh của từng thành viên và dùng các artefact như backlog, UML diagrams và prototype nhỏ để giữ cho dự án có cấu trúc rõ ràng. Việc chọn phạm vi vừa đủ, chỉ hiện thực một mẫu code minh họa thay vì cố xây dựng toàn bộ hệ thống, là phù hợp với yêu cầu của bài tập. Nhóm cũng nên tiếp tục sử dụng các tài liệu dùng chung để các thành viên dễ theo dõi tiến độ và đóng góp vào báo cáo.

Bảng 11: Tổng kết retrospective của nhóm theo khung Start / Stop / Continue

7.2 Lessons Learned

Qua dự án này, nhóm nhận ra rằng công nghệ phần mềm không bắt đầu từ code mà bắt đầu từ việc hiểu đúng vấn đề, xác định đúng phạm vi và giữ cho các quyết định thiết kế nhất quán với yêu cầu ban đầu. Một user story tốt cần được phản ánh trong acceptance criteria, trong sơ đồ UML, trong kiến trúc hệ thống, trong mẫu hiện thực và cuối cùng là trong test case. Nếu các phần này được viết tách rời, báo cáo có thể vẫn đầy đủ về hình thức nhưng thiếu một dòng chảy kỹ thuật rõ ràng. Scrum và backlog giúp nhóm chia nhỏ công việc, nhưng chỉ thật sự hiệu quả khi task ownership, thời điểm review và tiêu chí hoàn thành được thống nhất rõ. Nhóm cũng học được rằng việc không xây dựng một prototype hoàn chỉnh không phải là thiếu sót trong phạm vi bài này, vì yêu cầu của giảng viên cho phép concept, mock data và pseudo-code. Tuy nhiên, khi chọn hướng minh họa, nhóm vẫn cần trình bày trung thực ranh giới giữa thiết kế đầy đủ trên báo cáo và phần code sample đã hiện thực. Cải tiến quan trọng nhất cho dự án sau không phải là mở rộng phạm vi hệ thống, mà là làm rõ mối liên kết giữa yêu cầu, mô hình, mẫu hiện thực và kiểm thử ngay từ đầu.

7.3 Individual Reflections

Member	Role	Reflection
Nguyễn Văn Hùng	Implementation and Code Quality; Software Testing and Quality Assurance	Trong phần hiện thực và kiểm thử, em học được rằng việc tách validation thành các hàm riêng và suy nghĩ kỹ về edge cases ngay từ đầu giúp code dễ bảo trì và ít lỗi hơn. Nếu được làm lại, em sẽ thay các lệnh print bằng logging chuẩn, hiện thực trực tiếp phần code đã refactor thay vì chỉ mô tả, và bổ sung thêm white-box tests để kiểm tra trạng thái nội bộ của hệ thống.
Trần Minh Trí	Process Model and Planning; Architecture Design	Qua phần quy trình Scrum và thiết kế kiến trúc phân tầng, em học được cách lập kế hoạch linh hoạt và cách tách biệt các lớp để hệ thống dễ bảo trì hơn. Nếu có cơ hội làm lại, em sẽ phân tích sâu hơn cơ chế quét của Notification Worker để đảm bảo đáp ứng tốt hơn yêu cầu nhắc nhở trong vòng 60 giây của NFR1.
Nguyễn Tô Quốc Việt	Software Testing and Quality Assurance; Reflection and Retrospective	Trong quá trình tổng hợp retrospective, em nhận ra rằng việc phân chia nhiệm vụ trên bảng phân công là chưa đủ nếu nhóm không cập nhật rõ trạng thái thực hiện của từng phần. Khi phần testing đã được hỗ trợ bởi thành viên khác, em chuyển trọng tâm sang kiểm tra tính nhất quán của toàn bộ báo cáo và rút ra bài học rằng communication sớm quan trọng không kém bản thân việc chia task.
Lê Công Vinh	Requirements Engineering; System Modeling	Qua phần Requirements Engineering và System Modeling, em rèn luyện được kỹ năng tư duy thiết kế phần mềm và đảm bảo sự liên kết từ nhu cầu thực tế của người dùng đến mô hình hệ thống. Nếu được làm lại, em sẽ phân tích sâu hơn các kịch bản ngoại lệ trong State Diagram và Sequence Diagram để cải thiện khả năng xử lý lỗi của hệ thống.

Bảng 12: Phản ánh cá nhân của từng thành viên